

✓ DAA Tutorial

- ☐ DAA Tutorial
- ☐ DAA Algorithm
- ☐ Need of Algorithm
- ☐ Complexity of Algorithm
- ☐ Algorithm Design Techniques

✓ Asymptotic Analysis

- ☐ Asymptotic Analysis
- ☐ Analyzing Algorithm Control Structure

✓ Recurrence

- ☐ Recurrence Relation
- ☐ Recursion Tree Method
- ☐ Master Method

✓ Analysis of Sorting

- ☐ Bubble Sort
- ☐ Selection Sort
- ☐ Insertion Sort

✓ Divide and Conquer

- ☐ Introduction
- ☐ Max-Min Problem
- ☐ Binary Search
- ☐ Merge Sort
- ☐ Tower of Hanoi

✓ Sorting

- ☐ Binary Heap
- ☐ Quick Sort
- ☐ Stable Sorting

✓ Lower bound Theory

- ☐ Lower bound Theory

✓ Sorting in Linear Time

- ☐ Linear Time
- ☐ Counting Sort
- ☐ Bucket Sort
- ☐ Radix Sort

✓ Hashing

- ☐ Hashing
- ☐ Hash Tables
- ☐ Hashing Method
- ☐ Open Addressing Techniques
- ☐ Hash Function

✓ Binary Search Trees

- ☐ Binary Search

✓ Red Black Tree

- ☐ Red Black Tree

✓ Dynamic Programming

- ☐ Dynamic Programming
- ☐ Divide & Conquer Method vs Dynamic Programming
- ☐ Fibonacci sequence
- ☐ Matrix Chain Multiplication
- ☐ Matrix Chain Multiplication Example
- ☐ Matrix Chain Multiplication Algorithm
- ☐ Longest Common Sequence
- ☐ Longest Common Sequence Algorithm
- ☐ 0/1 Knapsack Problem
- ☐ DUTCH NATIONAL FLAG
- ☐ Longest Palindrome Subsequence
- ☐ Longest Increasing Subsequence
- ☐ Longest Common Subsequence
- ☐ Tabulation vs Memoization
- ☐ How to solve a dynamic programming problem
- ☐ Optimal Substructure Property
- ☐ Overlapping sub-problems

✓ Greedy Algorithm

ADVERTISEMENT

UP TO 38% OFF
ON FASTEST DEALS

BUY NOW

The Knuth-Morris-Pratt (KMP) Algorithm

← Prev

Next →

Knuth-Morris and Pratt introduce a linear time algorithm for the string matching problem. A matching time of $O(n)$ is achieved by avoiding comparison with an element of 'S' that have previously been involved in comparison with some element of the pattern 'p' to be matched. i.e., backtracking on the string 'S' never occurs

Components of KMP Algorithm:

1. The Prefix Function (Π): The Prefix Function, Π for a pattern encapsulates knowledge about how the pattern matches against the shift of itself. This information can be used to avoid a useless shift of the pattern 'p.' In other words, this enables avoiding backtracking of the string 'S.'

2. The KMP Matcher: With string 'S,' pattern 'p' and prefix function ' Π ' as inputs, find the occurrence of 'p' in 'S' and returns the number of shifts of 'p' after which occurrences are found.

The Prefix Function (Π)

Following pseudo code compute the prefix function, Π :

ADVERTISEMENT

BUY NOW

COMPUTE- PREFIX- FUNCTION (Π)

```
1. m ← length [P]           //'p' pattern to be matched
2.  $\Pi$  [1] ← 0
3. k ← 0
4. for q ← 2 to m
5. do while k > 0 and P [k + 1] ≠ P [q]
6. do k ←  $\Pi$  [k]
7. If P [k + 1] = P [q]
8. then k ← k + 1
9.  $\Pi$  [q] ← k
10. Return  $\Pi$ 
```

Running Time Analysis:

In the above pseudo code for calculating the prefix function, the for loop from step 4 to step 10 runs 'm' times. Step1 to Step3 take constant time. Hence the running time of computing prefix function is $O(m)$.

Example: Compute Π for the pattern 'p' below:

ADVERTISEMENT

GEL NAIL
POLISHBeats Powerbeats3 Wireless
Earphones - Black (Renewed)
Premium, Small
★★★★★ (2,427)
\$43.70 - \$45.99Mid 2019 Apple MacBook Pro with
1.6 GHz Intel Core i5 (13 inch, 8GB
RAM, 128GB SSD) Silver (Renewed)
★★★★★ (1,235)
\$480.47 - \$898.05Nintendo Switch Lite - Coral -
Switch (Renewed)
★★★★★ (2,298)
Nintendo Switch
\$175.99

BUY ON AMAZON

ADVERTISEMENT

- Dynamic programming vs Greedy approach
- Regular Expression Matching
- Branch and bound vs backtracking
- Branch and bound
- Longest Repeated Subsequence
- Longest Common Substring
- Shortest Common Supersequence
- Dynamic Programming vs Divide and Conquer
- Maximum Sum Increasing Subsequence
- Wildcard Pattern Matching
- Largest Sum Contiguous Subarray
- Shortest Sum Contiguous Subarray
- Dynamic programming vs Backtracking
- Brute force approach
- Fractional vs 0/1 knapsack problem
- Traveling Salesperson problem using branch and bound
- Integer Partition Problem
- Kruskal Algorithm
- Traveling Salesperson problem using branch and bound
- Greedy Algorithms
- Activity Selection Problem
- Fractional Knapsack problem
- Huffman Codes
- Algorithm of Huffman Code
- Activity or Task Scheduling Problem
- Travelling Sales Person Problem
- Dynamic Programming vs Greedy Method
- ✓ **Backtracking**
- Backtracking Introduction
- Recursive Maze Algorithm
- Hamiltonian Circuit Problems
- Subset Sum Problems
- N Queens Problems
- ✓ **MST**
- MST Introduction
- MST Applications
- Kruskal's Algorithm
- Prim's Algorithm
- ✓ **Shortest Path**
- Introduction
- Negative Weight Edges
- Representing Shortest Path
- Relaxation
- Dijkstra's Algorithm
- Bellman-Ford Algorithm
- Single Source Shortest Path in a directed Acyclic Graphs
- ✓ **All-Pairs Shortest Paths**
- Introduction
- Floyd-Warshall Algorithm
- Johnson's Algorithm
- ✓ **Maximum Flow**
- Flow networks and Flows
- Network Flow Problems
- Ford Fulkerson Algorithm
- Maximum bipartite matching
- ✓ **Sorting Networks**
- Comparison Network
- Bitonic Sorting Network
- Merging Network
- ✓ **Complexity Theory**
- Complexity Classes
- Polynomial Time Verification
- NP-Completeness
- Circuit Satisfiability
- 3-CNF Satisfiability
- Clique Problem
- Vertex Cover Problem

ADVERTISEMENT

P :

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Solution:

Initially: $m = \text{length}[p] = 7$
 $\Pi[1] = 0$
 $k = 0$

Step 1: $q = 2, k = 0$

$\Pi[2] = 0$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0					

Step 2: $q = 3, k = 0$

$\Pi[3] = 1$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0	1				

Step3: $q = 4, k = 1$

$\Pi[4] = 2$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0	1	2			

Step4: $q = 5, k = 2$

$\Pi[5] = 3$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0	1	2	3		

Step5: $q = 6, k = 3$

$\Pi[6] = 0$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0	1	2	3	0	

Step6: $q = 7, k = 1$

$\Pi[7] = 1$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0	1	2	3	0	1

After iteration 6 times, the prefix function computation is complete:

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
Π	0	0	1	2	3	0	1

The KMP Matcher:

The KMP Matcher with the pattern 'p,' the string 'S' and prefix function ' Π ' as input, finds a match of p in S. Following pseudo code compute the matching component of KMP algorithm:

KMP-MATCHER (T, P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\Pi \leftarrow \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4.  $q \leftarrow 0$  // numbers of characters matched
5. for  $i \leftarrow 1$  to  $n$  // scan S from left to right
```

UP TO 38% OFF ON
EASTER DEALS

BUY NOW



ADVERTISEMENT

UP TO 38% OFF
ON EASTER DEALS

BUY NOW

Subset-Sum Problem

Approximation Algo

- Introduction
- Vertex Cover
- Travelling Salesman Problem

String Matching

- Introduction
- Naive String Matching Algorithm
- Rabin-Karp-Algorithm
- String Matching with Finite Automata
- Knuth-Morris-Pratt Algorithm
- Boyer-Moore Algorithm

Misc

- Kosaraju Algorithm
- Hashing algorithm
- Dijkstra's Algorithm
- Huffman Coding Algorithm
- Kadane's Algorithm
- Dijkstra Algorithm Example
- Euclidean algorithm
- Floyd's Algorithm
- Properties of Algorithm
- Time Complexity of Kruskals Algorithm
- Travelling Salesman Problem
- Kosaraju's Algorithm
- Floyd-Warshall Algorithm
- Characteristics of an Algorithm
- Algorithm Examples
- Searching Algorithms
- Algorithm for Binary Search
- Sorting Algorithms: Slowest to Fastest
- Extended Euclidian Algorithm
- How to Write an Algorithm
- Recursive Algorithm
- Sliding Window Algorithm
- Difference Between Algorithms
- Flowcharts
- Ford Fulkerson Algorithm
- PageRank Algorithm
- Greedy Algorithm Example

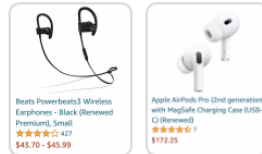
Interview Questions

- DAA Interview Questions

ADVERTISEMENT

```
6. do while q > 0 and P [q + 1] ≠ T [i]
    do q ← Π [q] // next character does not match
8. If P [q + 1] = T [i]
9. then q ← q + 1 // next character matches
10. If q = m // is all of p matched?
11. then print "Pattern occurs with shift" i - m
12. q ← Π [q] // look for the next match
```

ADVERTISEMENT

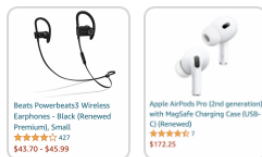


BUY ON AMAZON

Running Time Analysis:

The for loop beginning in step 5 runs 'n' times, i.e., as long as the length of the string 'S.' Since step 1 to step 4 take constant times, the running time is dominated by this for the loop. Thus running time of the matching function is O (n).

ADVERTISEMENT



BUY ON AMAZON

ADVERTISEMENT



Example: Given a string 'T' and pattern 'P' as follows:

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

ADVERTISEMENT



Let us execute the KMP Algorithm to find whether 'P' occurs in 'T.'

For 'p' the prefix function, π was computed previously and is as follows:

q	1	2	3	4	5	6	7
p	a	b	A	b	a	c	a
π	0	0	1	2	3	0	1

Solution:

Initially: n = size of T = 15
m = size of P = 7

ADVERTISEMENT

SAVE BIG WITH
EASTER DEALS



BUY ON AMAZON

ADVERTISEMENT

Step1: $i=1, q=0$

Comparing P [1] with T [1]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

P [1] does not match with T [1]. 'p' will be shifted one position to the right.

Step2: $i = 2, q = 0$

Comparing P [1] with T [2]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

P [1] matches T [2]. Since there is a match, p is not shifted.

Step 3: $i = 3, q = 1$

Comparing P [2] with T [3] P [2] doesn't match with T [3]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Backtracking on p, Comparing P [1] and T [3]

Step4: $i = 4, q = 0$

Comparing P [1] with T [4] P [1] doesn't match with T [4]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Step5: $i = 5, q = 0$

Comparing P [1] with T [5] P [1] match with T [5]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Step6: $i = 6, q = 1$

Comparing P [2] with T [6] P [2] matches with T [6]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Step7: $i = 7, q = 2$

Comparing P [3] with T [7] P [3] matches with T [7]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Step8: $i = 8, q = 3$

Comparing P [4] with T [8] P [4] matches with T [8]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Study Online:
Best Computer
Science courses

SHOFAST
EDUCATION



Step9: $i = 9, q = 4$

Comparing P [5] with T [9]

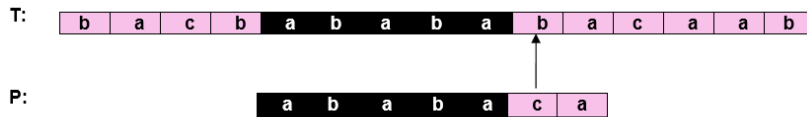
P [5] matches with T [9]



Step10: $i = 10, q = 5$

Comparing P [6] with T [10]

P [6] doesn't match with T [10]

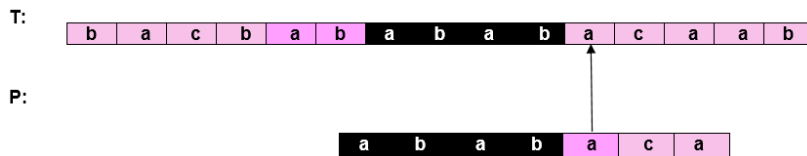


Backtracking on p, Comparing P [4] with T [10] because after mismatch $q = \pi [5] = 3$

Step11: $i = 11, q = 4$

Comparing P [5] with T [11]

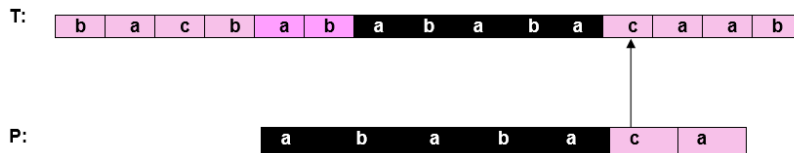
P [5] match with T [11]



Step12: $i = 12, q = 5$

Comparing P [6] with T [12]

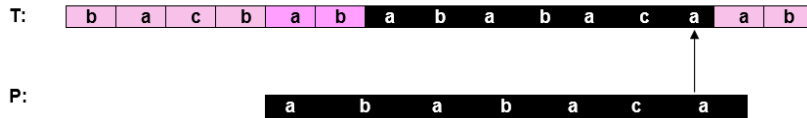
P [6] matches with T [12]



Step13: $i = 3, q = 6$

Comparing P [7] with T [13]

P [7] matches with T [13]



Pattern 'P' has been found to complexity occur in a string 'T.' The total number of shifts that took place for the match to be found is $i - m = 13 - 7 = 6$ shifts.

Next Topic [Boyer-Moore Algorithm](#)

← Prev

Next →

ADVERTISEMENT

×



 For Videos Join Our Youtube Channel: [Join Now](#)

Feedback

- Send your Feedback to feedback@javatpoint.com

ADVERTISEMENT

GEL NAIL POLISH



[BUY NOW](#)

ADVERTISEMENT

SAVE BIG WITH EASTER DEALS



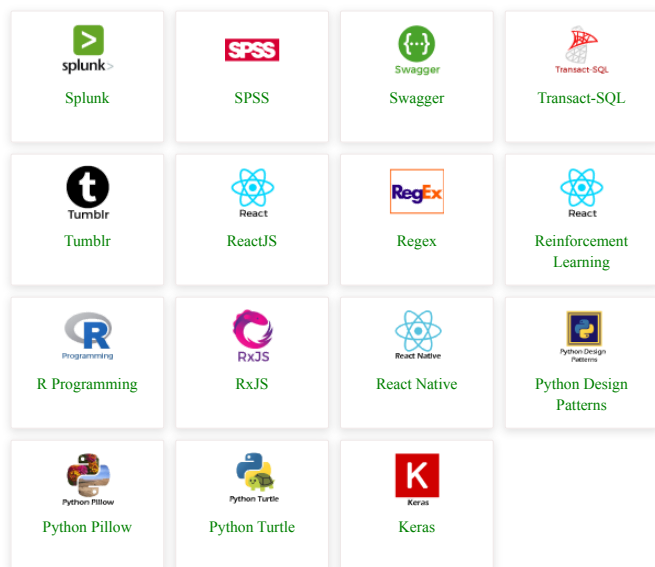
[BUY ON AMAZON](#)

ADVERTISEMENT

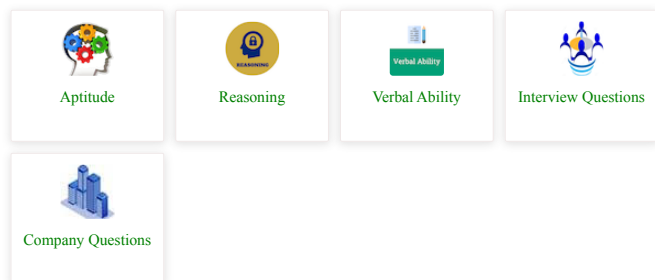
Help Others, Please Share



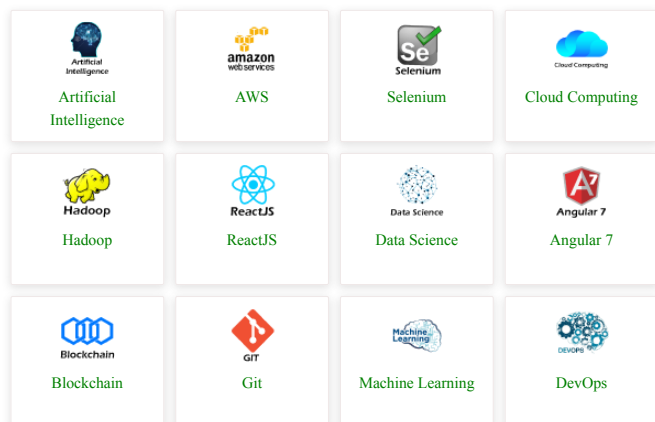
Learn Latest Tutorials



Preparation



Trending Technologies



ADVERTISEMENT

ADVERTISEMENT

SAVE BIG WITH EASTER DEALS

BUY NOW

B.Tech / MCA

 DBMS	 Data Structures	 DAA	 Operating System
 Computer Network	 Compiler Design	 Computer Organization	 Discrete Mathematics
 Ethical Hacking	 Computer Graphics	 Software Engineering	 Web Technology
 Cyber Security	 Automata	 C Programming	 C++
 Java	 .Net	 Python	 Programs
 Control System	 Data Mining	 Data Warehouse	

ADVERTISEMENT

SAVE BIG WITH EASTER DEALS

BUY NOW



ADVERTISEMENT

Like/Subscribe us for latest updates or newsletter



LEARN TUTORIALS

[Learn Java](#)
[Learn Data Structures](#)
[Learn C Programming](#)
[Learn C++ Tutorial](#)
[Learn C# Tutorial](#)
[Learn PHP Tutorial](#)
[Learn HTML Tutorial](#)
[Learn JavaScript Tutorial](#)
[Learn jQuery Tutorial](#)
[Learn Spring Tutorial](#)

INTERVIEW QUESTIONS

[Java Interview Questions](#)
[SQL Interview Questions](#)
[Python Interview Questions](#)
[JavaScript Interview Questions](#)
[Angular Interview Questions](#)
[Selenium Interview Questions](#)
[Spring Boot Interview Questions](#)
[HR Interview Questions](#)
[C++ Interview Questions](#)
[Data Structure Interview Questions](#)

ABOUT

Javatpoint is developed to help students on various technologies such as Artificial Intelligence, Machine Learning, C, C++, Python, Java, PHP, HTML, CSS, JavaScript, jQuery, ReactJS, Node.js, AngularJS, Bootstrap, XML, SQL, PL/SQL, MySQL etc.

Javatpoint provides tutorials with examples, code snippets, and practical insights, making it suitable for both beginners and experienced developers.

There are also many interview questions which will help students to get placed in the companies.

CONTACT

Address: G-13, 2nd Floor, Sec-3

Noida, UP, 201301, India

Contact No: 0120-4256464, 9990449935

[Contact Us](#)

[Subscribe Us](#)

[Privacy Policy](#)

[Sitemap](#)

[About Me](#)